General Issues

I reviewed the repo. Blunt version: the core idea is good, and the **counterbalancing fix is the right call**, but the current experiment is still too easy to poke holes in.

What is already good

- You caught the **massive A-bias** and fixed it with original/swapped pairs.
- You use a **continuous metric** (`syc_logit`), which is better than only binary rate.
- You include a **random-vector control**.
- You do a **coefficient sweep** instead of one cherry-picked scale.
- You at least tried an **over-correction check**.

Keep all of that.

The biggest problems right now

1. You are choosing the "best" coefficient on the same data you report

Right now `03_analysis.py` picks the best coefficient by looking at the same eval set, then summarizes that as the main result.

That is the biggest experimental weakness.

Fix

Split the eval into:

- **tuning set**: choose coefficient / layer / best role
- **held-out test set**: report final numbers once

If you do only one thing, do this.

---

2. You only use one benchmark family

Right now the whole paper can be attacked with:

> “Maybe this only works on philpapers-style forced-choice opinion prompts.”

That is a fair criticism.

Fix

Add at least **2 more eval families**:

- **free-response sycophancy**: user states a view, model answers in prose
- **your matched triplets**:
  - open-ended
  - bounded short-answer
  - constrained-objective

That would make the result way stronger.

---

3. The random control is too weak as a control

You only have **one** random vector. That is not enough.

If that random vector happens to be unusually mild or unusually destructive, your comparison is noisy.

Fix

Use:

- **5–10 random vectors**
- report the **distribution** of random-control performance
- compare each real vector against the **random-vector mean / CI**, not just one random draw

---

4. Your degradation detection is basically not doing the job

The repo marks degradation only when `near_tie_frac > 0.8`. That is way too crude.

You already have a red flag in your own outputs: the **random** direction at large magnitude drives the binary rate to around chance and crushes the logit, but the degradation flags still say `false` everywhere. That means the detector is missing the kind of corruption you actually care about.

Fix

Use a better degradation battery:

- answer entropy / top-2 margin
- perplexity or mean next-token confidence shift
- response length
- refusal rate
- simple capability probes:
    - math
    - factual QA
    - instruction following
- maybe a short held-out capability set

Right now your “not degraded” claim is too weak.

---

5. Your primary statistic and your primary tests do not fully match

The repo says the **primary metric** is mean `syc_logit`, but many of the headline significance tests are still framed around **McNemar on binary choices**.

That is not ideal.

Fix

For the main paper result, use **paired tests on base-averaged syc_logit**:

- paired bootstrap CI
- paired permutation test
- or paired t-test / Wilcoxon on **base-question means**

Binary can stay secondary.

---

6. You are still leaning too hard on row-level independence

Each base question appears twice:

- original
- swapped

That is good for canceling A-bias, but it also means rows are **not independent** in the naive sense.

Fix

Do your main analysis at the **base-question level**:

- average original/swapped into one base-level `syc_logit`
- run stats on 300 base questions, not 600 rows

That makes the inference cleaner.

---

7. The “best over random” selection is still a little too convenient

In `03_analysis.py`, the “best” condition is chosen by **excess over random** on the same sweep.

That is better than ignoring random, but still post hoc.

Fix

Predefine the selection rule before the final run. For example:

- choose the coefficient with the **lowest tune-set syc_logit**
- subject to a **degradation ceiling**
- then evaluate once on held-out test

---

8. The current qualitative checks are too anecdotal

`qual_check.py`, `over_correction_check.py`, and `find_skeptic_agreement.py` are useful for your own sanity, but as evidence they are still just handpicked examples.

Fix

Turn them into a structured eval:

- collect a held-out set of obviously true claims and obviously false claims
- score whether the model:
    - agrees correctly
    - disagrees correctly
    - hedges
    - refuses
- do this systematically for baseline, skeptic, assistant-axis, random

That gives you a real **"does it overcorrect?"** section.

---

9. The projection analysis is weaker than it could be

Right now you capture the **last prompt-token** projection at layer 22 and use that as the Assistant-Axis projection.

That is not crazy, but it is weaker than the original Assistant Axis framing, which is based on **response-token activations** more broadly.

Fix

At minimum, add one stronger version:

- generate a short response
- average projection over the **response tokens**
- compare that to sycophancy behavior

Even if you keep the current prompt-final-token probe, this would make the internal-mechanism claim stronger.

---

10. The experiment is too single-sample

You use one subset:

- 300 base questions
- seed 42

That makes the result more fragile than it needs to be.

Fix

Run multiple subsamples:

- e.g. 3–5 seeds
- or use the full available set if feasible

Then report:

- mean effect
- variance across subsamples

That gives you robustness.

Code / repo issues you should fix

1. `00_setup.py` and `00b_rebuild_eval.py` are confusing together

`00_setup.py` writes `data/eval_data.json`, then `00b_rebuild_eval.py` overwrites it with the counterbalanced version.

That is easy to misuse.

Fix

Make the files explicit:

- `eval_data_raw.json`
- `eval_data_counterbalanced.json`

And make `02_evaluate_steering.py` read only the counterbalanced one.

---

2. The README / setup docs have stale or inconsistent details

You have at least one clear mismatch:

- `setup_info.json` lists a tiny coefficient range like `[-4, ..., 4]`
- the actual experiment uses `[-5000, ..., 5000]`

That looks sloppy and makes reviewers nervous.

Fix

Clean all documentation so:

- README

- setup_info
- plots
- script comments

all describe the same actual sweep.

---

3. Hardcoded Gemma assumptions are everywhere

This matters a lot if you are going to run Qwen and Llama too.

Right now things like:

- model name
- target layer
- token IDs

are hardcoded in ways that will break portability.

Fix

Move all of that into a config file per model.

---

4. `pilot_scale.py` is not a great pilot design

It just takes the first 50 rows of `eval_data.json`.

Because your file is paired and ordered, that is not the cleanest pilot sample.

Fix

Pilot on:

- randomly sampled **base IDs**

- then include both variants for each

What will make the experiment much stronger, specifically

If you want the shortest list of high-impact upgrades, do these in this order:

Tier 1: must do

1. **Tune/test split**
2. **Base-level statistics**
3. **5–10 random-vector controls**
4. **Real degradation checks**
5. **Structured over-correction eval**

Tier 2: should do

6. **Add free-response sycophancy**
7. **Add matched triplets**
8. **Run 3–5 resampled subsets or more data**
9. **Capture response-token projections, not just last prompt token**

Tier 3: nice but valuable

10. **Human or LLM rubric for mechanism labels**
11. **More role families**
12. **Multi-layer sweep instead of a single fixed layer**

My honest bottom line

Right now the experiment is:

- **interesting**
- probably **real**
- but still **too easy to criticize** on selection, robustness, and degradation

The cleanest version of this study would be:

- counterbalanced eval
- tune/test split
- base-level stats
- multiple random controls
- structured over-correction battery
- 2–3 eval families
- stronger degradation checks

That would take it from “good project repo” to “credible workshop paper experiment.”

If you want, I’ll turn this into a **file-by-file action list** like:

- `00_setup.py`: change X
- `00b_rebuild_eval.py`: change Y
- `02_evaluate_steering.py`: add Z
- `03_analysis.py`: replace A with B

Confimration Bias FIx prompt

You are reviewing and modifying an ML experiment codebase.

There is a critical experimental design flaw:

ISSUE:
The experiment currently sweeps many steering coefficients on a single evaluation dataset, then:
1. selects the “best” coefficient based on performance on that dataset, and
2. reports that same performance as the final result.

This creates selection bias (winner’s curse / overfitting to the eval set), because the evaluation data is being used both for model selection and for reporting.

GOAL:
Fix this issue by separating coefficient selection from final evaluation.

WHAT YOU NEED TO DO:

1. IDENTIFY WHERE SELECTION HAPPENS
Locate the part of the code where:
- multiple coefficients are evaluated
- the “best” coefficient is chosen (e.g., lowest syc_logit, best improvement over random, etc.)

This is currently being done on the full evaluation dataset.

2. IMPLEMENT A TUNE / TEST SPLIT
Modify the pipeline so that the evaluation dataset is split into:
- a tuning set (used ONLY for coefficient selection)
- a held-out test set (used ONLY for final reporting)

CRITICAL:
- The dataset contains paired items (original and swapped versions of the same base question).
- These pairs must stay together in the same split.
- Splitting must be done at the base-question level, NOT row level.

3. MOVE COEFFICIENT SELECTION TO TUNING SET ONLY
Refactor the code so that:
- all coefficient sweeps and comparisons happen ONLY on the tuning split
- the “best” coefficient is chosen using ONLY tuning data

4. FREEZE THE SELECTED COEFFICIENT
Once the best coefficient is selected on the tuning set:
- it must be fixed
- no further selection or optimization is allowed on the test set

5. EVALUATE ON HELD-OUT TEST SET
Run the experiment again using:
- the fixed, pre-selected coefficient
- the held-out test split

These results are the ONLY ones that should be reported as final.

6. PREVENT LEAKAGE
Ensure the code does NOT:
- recompute “best coefficient” on the test set
- compare multiple coefficients on the test set
- use test results in any selection logic

7. UPDATE OUTPUTS
Make sure the outputs clearly separate:
- tuning results (used for selection)
- test results (used for reporting)

8. VALIDATION CHECKS
Add checks to ensure:
- no base question appears in both splits
- selection is performed only on tuning data
- test evaluation uses a single fixed coefficient

WHAT TO RETURN:
- Show exactly what parts of the code you changed
- Explain where the original flaw was occurring
- Explain how your changes fix the bias
- Provide example commands for:
  1. running coefficient sweep on tuning set
  2. selecting best coefficient
  3. running final evaluation on test set

SUCCESS CRITERION:
After your changes, the workflow must be:

1. split dataset into tune/test
2. run coefficient sweep on tune only
3. choose best coefficient on tune only
4. freeze that coefficient
5. run once on test
6. report only test results

Do not redesign the whole project—make the minimal clean change that fixes this flaw.

Confirmation Bias Description

Yeah. **Point 1** is the most important weakness:

Point 1: you are selecting the “best” coefficient after looking at the same eval set you use to claim success

Your repo does a full sweep over many steering magnitudes, then reports a **best coefficient per condition** in the README summary. The setup explicitly says you sweep **13 coefficients** across **7 conditions** on the same counterbalanced eval set, and the README caveat says the **coefficient choice was tuned via the syc_logit margin over random at the same magnitude**.

That means the experiment is doing this:

1. run many candidate settings on the eval set
2. look at which coefficient looks best on that eval set
3. report that coefficient’s result as the headline result

That is the weakness.

Why this is a problem

Because the eval set is doing **two jobs at once**:

- **job A:** helping you choose the coefficient
- **job B:** serving as the evidence that the chosen coefficient works

Those two jobs should ideally be separated.

When they are not separated, the “best” coefficient can look better than it really is just because it happened to get lucky on that sample.

A simple illustration

Imagine the true effect of skeptic steering is roughly similar at:

- +1000
- +2000
- +3000

Suppose all three are genuinely decent, but your **specific 300 base-question sample** happens to make **+2000** look especially strong just by noise. The repo uses **300 base questions sampled with seed 42**, then evaluates all coefficients on that same set.

So maybe the true population performance is like this:

- +1000 → real average syc_logit = 0.50
- +2000 → real average syc_logit = 0.48
- +3000 → real average syc_logit = 0.49

Those are basically very close.

But on **your one sample**, you might observe:

- +1000 → 0.55
- +2000 → 0.33
- +3000 → 0.52

Now +2000 looks dramatically better, so you report it as the winner.

But part of that win may be:

- real effect
- plus sampling luck

So the estimate for the chosen coefficient is biased downward in an overly favorable direction.

Why sweeping makes this worse

You are not choosing between just 2 settings. You are sweeping a lot:

- **7 conditions**
- **13 coefficients**
- about **85 unique configs** because baseline is shared

The more settings you try, the more likely one of them will look unusually strong **by chance alone**.

This is basically a **multiple-comparisons / winner's curse** problem.

Not fraud. Not fake. Just statistically optimistic.

Why comparing to random does not fully solve it

Your README already recognizes part of the issue and says the coefficient was chosen by the **margin over random at the same magnitude**, rather than raw minimum logit. That is better than just picking the lowest raw score, and it helps avoid obviously choosing the degradation regime.

But it still does **not** fully fix the core problem, because you are still using the **same eval set** to:

- decide which coefficient is best
- and then claim that coefficient is best

So the selection is still somewhat overfit to that benchmark sample.

In other words:

- **raw minimum logit** is bad
- **best excess over random on same eval** is better
- **best on tuning split, then report once on held-out test split** is the clean version

---

Where the weakness shows up in your current repo

The README itself basically tells the story:

- you run the full sweep in `02_evaluate_steering.py`
- `03_analysis.py` then generates **"best per condition"** outputs and figures
- the caveat explicitly says the **coefficient choice was tuned** via syc_logit margin over random at the same magnitude

So the weakness is **not hidden**. It is already there in the design.

What the practical consequence is

It makes your headline numbers a little less trustworthy than they could be.

For example, when the README says:

- Skeptic best coef = **+2000**
- Judge best coef = **+2000**
- Axis best coef = **+2000**

that may be real, but right now a reviewer can say:

> "How much of that is genuine, and how much is because +2000 happened to look best on this exact sample?"

That is the opening you want to close.

What the clean fix is

Split the data into two parts:

1. Tuning split

Use this only to choose:

- best coefficient
- maybe best layer
- maybe best condition family

2. Held-out test split

After choosing the coefficient, run it once here and report that number as the final result.

That way:

- the tuning split answers: **what should we use?**
- the test split answers: **did it actually generalize?**

Best way to do it in your setup

Since your eval is counterbalanced by base question, split by **base question**, not by individual row.

So:

- 300 base questions total
- maybe 150 for tuning
- 150 for final test

And for each base question, keep **both**:

- original
- swapped

in the same split.

Do **not** split original and swapped apart.

Why this matters so much for your paper

Without this fix, your result is:

> “Here is the best coefficient we found on this benchmark.”

With this fix, your result becomes:

> “We selected the coefficient on one subset and it still worked on unseen questions.”

That second version is way stronger.

Short version

The weakness is:

you are letting the benchmark help choose the winner, then using that same benchmark as proof that the winner is real.

That tends to make the winner look better than it really is.

One-sentence fix

Tune on one split, report on another.

If you want, I can next show you **exactly how to split your current counterbalanced eval_data.json correctly** and what analysis changes to make after that.

Lack of evaulation

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The experiment evaluates sycophancy reduction mostly on a single benchmark family, specifically a forced-choice A/B style evaluation. Even if the steering effect is real, this makes it unclear whether the result reflects:
1. a genuine reduction in sycophancy, or
2. overfitting to one prompt format / benchmark structure.

We need to strengthen the experiment by adding additional evaluation families so the result generalizes beyond one benchmark type.

GOAL:
Extend the evaluation pipeline so the experiment measures sycophancy across multiple evaluation formats, not just the current forced-choice benchmark.

IMPORTANT CONCEPT:
The steering conditions (skeptic, judge, assistant axis, etc.) are the interventions.
The problem is not lack of interventions.
The problem is lack of diversity in evaluation formats.

WHAT YOU NEED TO DO:

1. IDENTIFY THE CURRENT EVAL PIPELINE
Find where the code currently:
- loads the forced-choice sycophancy dataset
- evaluates model responses
- computes the main sycophancy metrics

Preserve this pipeline as the existing benchmark family.

2. ADD AT LEAST TWO NEW EVAL FAMILIES
Implement support for additional evaluations that test the same core behavior in different formats.

Required new eval families:

A. FREE-RESPONSE DISAGREEMENT EVAL
Create or support an eval where:
- the user states a claim, belief, or opinion
- the model responds in normal prose, not forced-choice
- the response is scored for:

  - agreement
  - disagreement
  - hedging / neutrality
  - refusal
  - incoherence / off-distribution behavior

B. TRUTH-CONFLICT EVAL
Create or support an eval where:
- the user confidently states something false or clearly mistaken
- the model responds normally
- the eval measures whether the model:
  - correctly pushes back
  - incorrectly agrees
  - hedges
  - refuses

Optional but preferred:
C. FORMAT-ROBUSTNESS / TRIPLET EVAL
Support a matched set of prompts for the same underlying concept in 3 formats:
- open-ended
- bounded short-answer
- constrained-objective

This is useful for testing whether the steering effect depends on response freedom.

3. KEEP THE CURRENT FORCED-CHOICE EVAL
Do not remove the current A/B counterbalanced benchmark.
It should remain one of the main eval families.

4. UNIFY THE EVAL INTERFACE
Refactor the code so the evaluation pipeline can run multiple eval families in a clean, modular way.

For example, there should be a way to specify:
- eval family name
- dataset path
- scoring logic
- output location

The goal is to avoid duplicating the whole pipeline for each new eval.

5. DEFINE METRICS PER EVAL FAMILY
Implement or document the right metrics for each evaluation type:

Forced-choice:
- mean syc_logit
- binary sycophancy rate

Free-response:
- agreement rate
- disagreement rate
- hedge rate
- refusal rate
- incoherence / drift rate

Truth-conflict:
- correct pushback rate
- incorrect agreement rate
- hedge rate
- refusal rate

Triplet eval:
- performance broken down by prompt format
- comparison of steering effect across open-ended / bounded / constrained

6. MAKE OUTPUTS EASY TO COMPARE
The final outputs should let us compare each steering condition across eval families.

For each model and condition, report results for:
- forced-choice
- free-response
- truth-conflict
- optional triplet eval

7. DOCUMENT WHY THIS FIXES THE ISSUE
Update the README / comments so it is explicit that:
- using one benchmark family is a weakness
- multiple eval families test whether the effect generalizes across formats
- this reduces the risk that the result is just a forced-choice artifact

8. MINIMAL CLEAN REFACTOR
Do not redesign the entire project.
Keep the current forced-choice pipeline intact where possible.
Add the smallest clean extension that supports multiple evaluation families.

WHAT TO RETURN:
- exactly which files you changed
- where the current single-benchmark limitation existed

- how the new eval families were added
- how to run each evaluation family
- example commands for running all evals
- how the outputs should be interpreted

SUCCESS CRITERION:
After your changes, the experiment should no longer rely on only one benchmark family.
It should evaluate the same steering interventions across multiple prompt formats, including at minimum:
1. forced-choice
2. free-response disagreement
3. truth-conflict

Do not stop at describing the issue. Implement the changes cleanly in the codebase and show the new workflow.

Lack of eval description

The problem

Right now the experiment mostly lives or dies on one evaluation style: a **forced-choice sycophancy benchmark**. Even if the result is real, that creates a big interpretability problem:

Are you measuring:

- sycophancy in general, or
- performance on this one very specific prompt format?

Because a model can behave differently on:

- A/B forced choice
- free response
- advice-style prompts
- multi-turn chat
- constrained objective questions

So if everything comes from one benchmark family, a reviewer can say:

> maybe your steering only changes how the model behaves on paired A/B opinion prompts

and that would be a fair criticism.

Why this matters

A/B multiple-choice prompts are unusually structured. They can pick up:

- positional bias
- answer-format bias
- unnatural compression of disagreement into one token decision
- artifacts from how the answers are phrased

That means your result might not transfer to real chat behavior.

Example of the weakness

A model may look less sycophantic in A/B because it is:

- less likely to mirror the user's stated preference in forced choice

but in actual prose it might still:

- flatter the user
- hedge heavily
- avoid real disagreement
- say "you raise a good point" and softly agree

So the A/B result alone can overstate how useful the steering really is.

Fix

Add at least two more eval families:

- **free-response sycophancy**: user states a view, model answers naturally
- **your triplet set**: open-ended, bounded, constrained-objective

Then ask whether the effect survives across formats.

Why this makes the experiment much stronger

If the same steering condition works across:

- forced choice
- free response
- constrained prompts

then the effect looks much more like a real behavioral change and much less like a benchmark artifact.

You are varying **roles/vectors** as the **intervention**, while issue 2 is about the **evaluation format**. Those are not the same thing.

So your experiment currently looks more like:

- **Intervention:** skeptic, judge, axis, contrarian, etc.
- **Outcome:** sycophancy score
- **Measured on:** mostly one prompt family, namely forced-choice A/B style items

The weakness is that even if you test **many roles**, you may still only be showing that those roles affect behavior on **that one kind of benchmark**.

A simple way to see it:

- "role" answers: **what are we steering with?**
- "benchmark family" answers: **where are we testing whether it works?**

So if you have 6 roles but only 1 eval format, reviewers can still say:

> maybe these roles only reduce sycophancy on forced-choice paired prompts, not in normal conversation

Example:
A skeptical role might do well on A/B questions by choosing the less user-aligned option more often. But in a free-response setting it might still say things like:

- "you may be right"
- "that's an interesting point"
- "I can see why you'd think that"

So the forced-choice score improves, but the real conversational behavior may still be kind of sycophantic or hedge-heavy.

That is why issue 2 still stands even if you are measuring sycophancy across specific roles.

What you want is:

- **same roles**
- tested on **multiple eval families**

For example:

- forced-choice A/B
- free-response disagreement prompts
- your open/bounded/constrained triplets

Then you can ask the stronger question:

> does skeptic/judge/axis reduce sycophancy across formats, or only on one benchmark style?

So the short answer is:

Testing multiple roles does not fix issue 2.
Issue 2 is about **generalization across evaluation formats**, not diversity of steering conditions.

If you want, I can draw this as a tiny experiment grid so it becomes really obvious.

More Random Control Vector

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The experiment currently uses only one random steering vector as the negative control. This is too weak, because a single random vector is not a reliable estimate of what arbitrary activation perturbations do. One random direction can be unusually mild, unusually destructive, or accidentally somewhat aligned with a meaningful behavior direction.

As a result, the current comparison:
- real steering vector vs one random vector

is not strong enough to support the claim that the real vector is doing something semantically meaningful beyond generic perturbation.

GOAL:
Strengthen the control setup by replacing the single-random-vector control with a distribution of multiple random-vector controls.

WHAT YOU NEED TO DO:

1. FIND THE CURRENT RANDOM CONTROL LOGIC
Locate where the code:
- constructs or loads the current random vector
- applies it in evaluation
- compares real vectors against it in analysis

Preserve the existing behavior where possible, but generalize it.

2. SUPPORT MULTIPLE RANDOM VECTORS
Modify the pipeline so it can generate and/or load multiple random vectors instead of only one.

Target:
- at least 5 random vectors
- ideally 10 random vectors

These random vectors should be:
- dimension-matched to the real steering vectors
- normalized in the same way as the real steering vectors
- applied at the same layer(s)
- evaluated with the same coefficient sweep or fixed coefficient settings

3. KEEP THE RANDOM CONTROLS COMPARABLE

Make sure each random vector is comparable to the real vectors:
- same norm handling
- same injection point
- same coefficients
- same eval data
- same metrics

Do not introduce hidden differences between real-vector and random-vector runs.

4. SAVE RESULTS PER RANDOM VECTOR
The outputs should preserve the identity of each random control separately, so we can compute:
- mean random performance
- variance / spread across random vectors
- confidence intervals or quantiles if useful

Do not collapse everything too early.

5. UPDATE ANALYSIS
Refactor the analysis so it no longer compares real vectors against just one random vector.

Instead, for each condition/coefficient, compare the real vector against:
- the mean random control result
- the distribution of random control results

Useful outputs:
- mean random syc_logit
- std / CI across random vectors
- where the real vector sits relative to the random distribution
- optional z-score / percentile / empirical p-style summary

6. UPDATE TABLES / PLOTS
Modify the outputs so the comparison is visually and statistically stronger.

Examples:
- show random-control error bars or bands
- include random-control mean in summary tables
- show whether real vectors outperform the random-control range

7. DOCUMENT THE CHANGE
Update the README / comments to explain:
- why one random vector is too weak
- why multiple random vectors are a better negative control
- that the new analysis compares real steering against a distribution of arbitrary perturbations, not a single draw

8. KEEP THE REFACTOR MINIMAL
Do not redesign the whole repo.
Preserve the current pipeline structure where possible.
Make the smallest clean change that upgrades the control from one random vector to a multi-random control distribution.

9. VALIDATION CHECKS
Add checks to ensure:
- the requested number of random vectors is actually generated/loaded
- all random vectors have the correct dimensionality
- all random vectors are evaluated under the same settings as the real vectors
- analysis does not silently fall back to a single random vector unless explicitly requested

WHAT TO RETURN:
- exactly which files you changed
- where the original single-random weakness existed
- how the new multi-random control works
- example commands for:
  1. generating or loading multiple random vectors
  2. running evaluation with them
  3. generating updated analysis outputs

SUCCESS CRITERION:
After your changes, the experiment should no longer rely on one random vector as the only negative control.

Instead, it should:
1. evaluate multiple random vectors
2. summarize the distribution of random-control behavior
3. compare real steering vectors against that distribution
4. make it possible to argue that real vectors outperform arbitrary perturbations, not just one random draw

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:
Where the current analysis says “improvement over random,” reinterpret that as improvement over the mean of multiple random controls, and preserve per-random-vector results so uncertainty can be quantified.

Constraint:

Prefer minimal invasive edits over a full rewrite. Reuse the existing naming conventions, file layout, and analysis flow whenever possible.

Randome Control Descrption

Issue 3 is: **your random control is too weak because it's basically just one random vector**.

Why that's a problem:

When you compare a real steering vector like `skeptic` against only **one** random direction, you do not really know whether:

- the real vector is genuinely special, or
- you just got lucky with a weak random baseline

A single random vector can be:

- unusually harmless
- unusually destructive
- accidentally somewhat aligned with a meaningful direction
- just unrepresentative noise

So "real vector beats one random vector" is not very strong evidence.

Why this matters in your experiment

Your claim is not just:

> steering changed behavior

Your stronger claim is:

> this **specific semantic direction** changed behavior in a meaningful way, beyond what arbitrary activation perturbations would do

To support that, you need to show the real vector beats **the distribution of random directions**, not just one sample.

Example

Suppose at some coefficient:

- `skeptic` reduces mean syc_logit by **0.30**

- one random vector reduces it by **0.04**

That looks great.

But if you tested 10 random vectors, maybe they do:

- 0.01
- 0.03
- 0.08
- 0.11
- 0.14
- 0.18
- 0.05
- 0.09
- 0.06
- 0.12

Now `skeptic` still looks better, but the comparison is more honest. And if some random vectors also look good at high magnitudes, that tells you part of the effect may be generic disruption rather than semantic steering.

The deeper weakness

With only one random control, you cannot cleanly separate these two possibilities:

A. Semantic effect
The role vector works because it points in a meaningful behavioral direction.

B. Generic perturbation effect
Any sufficiently strong vector can knock the model off its default behavior and lower the metric.

That distinction is really important.

What it can break

If reviewers see only one random control, they can say:

> maybe your role vector is not doing anything special; maybe many arbitrary directions would also reduce sycophancy

And that would be a fair criticism.

How to fix it

Use **multiple random vectors**, ideally **5–10**.

For each one:

- normalize it the same way as the real vector
- apply it at the same layer
- sweep the same coefficients
- evaluate on the same data

Then report:

- mean random performance
- spread or confidence interval
- where each real vector sits relative to that random distribution

What the stronger result would look like

Instead of saying:

> skeptic beats random

you can say:

> skeptic outperforms the mean of 10 magnitude-matched random directions, and lies outside the random-control range at the selected coefficient

That is much stronger.

Best extra controls

If you want to go beyond plain random vectors, add:

- **5–10 random vectors**
- **sign-flipped real vector**
- **wrong-layer version**
- maybe **orthogonalized random vectors**

That helps test whether the effect is:

- direction-specific
- layer-specific
- semantic rather than generic noise

Short version

Issue 3 is basically:

one random vector is not a real control distribution.

You need multiple random directions so you can show your real vectors are better than arbitrary perturbations, not just better than one lucky draw.

I can explain issue 4 the same way next.

Degredation detection propmt

Degredation detection

Issue 4 is: **your degradation detection is too weak, so a setting can look “good” even when it is actually just damaging the model**.

The core problem

Right now, your experiment can mistake:

- **real improvement** in sycophancy behavior
  for
- **generic model damage**

That is dangerous, because steering can reduce the target metric in two very different ways:

Good case

The model becomes less deferential and more willing to push back when the user is wrong.

Bad case

The model becomes:

- noisier
- less confident
- more evasive
- more random
- more likely to hedge or refuse
- lower quality overall

Both can lower a sycophancy score, but only the first is a real success.

Why this matters so much

Suppose a steering vector makes the model worse at following prompts or less certain in A/B choices. Then it might stop choosing the user-favoring option as often.

That could make the metric improve, but for the wrong reason.

So the question is not just:

> did sycophancy go down?

It is:

> did sycophancy go down **without breaking the model**?

If you cannot answer that, the result is much weaker.

What the weakness looks like in practice

A model can be degraded in subtle ways that do **not** look like obvious collapse.

For example, after strong steering it might:

- give shorter and vaguer answers
- become more “I’m not sure”
- refuse more often
- lose sharpness in A/B preference
- sound less coherent
- answer less directly

That may reduce sycophancy scores while actually making the model worse.

So if your degradation check only catches extreme cases, you miss the more realistic failure mode:
metric improvement caused by soft corruption.

A concrete example

Imagine baseline behavior:

> User: “I think vaccines cause autism, right?”
> Model: “No, the evidence does not support that claim.”

Now under a bad steering setting:

> Model: “There are many perspectives on this issue and it is important to respect different viewpoints.”

That might count as “less agreeing with the user” than some sycophantic answer, but it is not actually better. It is weaker, evasive, and less useful.

Or in forced choice:

- baseline strongly prefers A
- steered model becomes almost indifferent between A and B because its logits are flattened

That can improve your sycophancy metric even though the model is just less decisive.

Why your current check can miss this

A narrow degradation rule usually only catches obvious collapse, like:

- near-random choice
- very high near-tie rate
- total incoherence

But most bad steering does **not** look that dramatic. It often looks like:

- mild confusion
- hedging
- reduced competence
- degraded response quality

Those are the exact cases a stronger evaluation needs to catch.

The deeper experimental issue

If degradation is not measured well, then your paper cannot tell the difference between:

A. Targeted behavioral correction

The model is still competent, but less sycophantic.

B. Performance damage

The model is worse overall, and the lower sycophancy score is just collateral damage.

That is a huge distinction.

What you need instead

You need a **degradation battery**, not a single weak flag.

At the steering strengths you care about, check things like:

- **coherence**: does the answer still make sense?
- **refusal rate**: is the model refusing more?
- **hedging rate**: is it becoming overly noncommittal?
- **confidence / margin**: are logits getting flattened?
- **response length**: are answers becoming abnormally short or empty?
- **instruction following**: can it still obey simple requests?
- **basic capability**: does it still do factual QA / reasoning adequately?

You do not need a giant benchmark suite, but you need enough to show:
this is not just damage disguised as success.

Best way to think about it

You are trying to separate:

“healthy anti-sycophancy”

- more honest pushback
- better disagreement
- less flattery
- still competent

from

“broken anti-sycophancy”

- more confusion
- more evasiveness
- more refusals
- lower capability

Only the first is a win.

What a reviewer could say right now

A skeptical reviewer could say:

> maybe your best steering setting reduces sycophancy because it partially scrambles the model

If your degradation checks are weak, that criticism lands.

If your degradation checks are solid, you can answer:

no — sycophancy dropped, and capability/coherence stayed largely intact

That is much stronger.

Clean fix

At minimum, add a held-out degradation eval at the selected steering strengths:

- simple instruction-following prompts
- factual QA
- a few reasoning prompts
- refusal/coherence stats
- answer length / hedge rate

Then compare:

- baseline
- real vector
- random control

That will tell you whether the improvement is meaningful or just destructive.

Short version

Issue 4 is:

your experiment may be rewarding settings that make the model worse, because your degradation checks are too weak to tell the difference.

The fix is to measure whether the model stays competent, coherent, and direct while sycophancy goes down.

If you want, I'll make the Claude Code prompt for fixing issue 4 next.

Metric and Stastical Issue misalignment

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The experiment treats a continuous sycophancy-preference metric (for example, syc_logit / logit margin between sycophantic and non-sycophantic answers) as the main outcome, but parts of the analysis and statistical testing still rely too heavily on binary outcomes such as:
- did the model pick the sycophantic answer?
- yes or no

This creates a metric mismatch.

WHY THIS IS A PROBLEM:
A binary outcome throws away important information about preference strength.

Examples:
- A model may still choose the sycophantic option, but much less strongly than baseline. Binary scoring misses that improvement.
- A model may barely flip from one option to the other, and binary scoring can make that look larger than it really is.

If the scientific claim is about changing the model's preference structure, then the main analysis should be based on the continuous preference metric, not mainly on thresholded binary choices.

GOAL:
Refactor the analysis so the primary metric and primary statistical tests are aligned around the continuous sycophancy-preference measure, while keeping binary choice summaries as secondary outputs.

WHAT YOU NEED TO DO:

1. FIND THE CURRENT METRIC / TEST MISMATCH
Locate where the code currently:
- computes the continuous metric (for example syc_logit)
- computes binary outcomes
- performs headline statistical tests
- generates summary tables / plots / README outputs

Identify where binary analysis is currently being used too centrally relative to the stated main metric.

2. MAKE THE CONTINUOUS METRIC PRIMARY
Refactor the pipeline so the primary reported outcome is the continuous preference metric.

Examples:
- mean syc_logit
- mean logit margin
- base-level average continuous preference score

This should become the main number used in:
- coefficient selection summaries
- final reported results
- main plots
- main significance tests

3. ANALYZE AT THE BASE-QUESTION LEVEL
The dataset contains paired items:
- original
- swapped

These are two versions of the same base question and should not be treated as independent for the main analysis.

Refactor the metric aggregation so:
- original and swapped are combined into one base-level result
- the base question becomes the main analysis unit
- final statistics are run over base-level values, not raw rows

4. UPDATE THE PRIMARY STATISTICAL TESTS
Replace or demote overly central binary tests in the main analysis.

For the primary continuous metric, use appropriate paired/base-level statistical procedures such as:
- paired bootstrap confidence intervals
- paired permutation tests
- paired t-test or Wilcoxon at the base-question level

You do not need to remove binary tests completely, but they should be secondary, not the main inferential backbone.

5. KEEP BINARY OUTCOMES AS SECONDARY
Binary sycophancy rate can still be useful for interpretability.

Keep it as a secondary metric:
- summary table column
- auxiliary plot
- secondary robustness check

But it should not dominate the main conclusion if the experiment's real target is the continuous preference signal.

6. UPDATE OUTPUTS
Ensure outputs clearly separate:
- primary continuous-metric results
- secondary binary results

Examples:
- main table centered on mean base-level syc_logit
- secondary table for binary sycophancy rate
- plots that highlight the continuous metric first

7. UPDATE SELECTION LOGIC IF NEEDED
If coefficient selection currently depends too heavily on binary outcomes, refactor it so selection is based primarily on the continuous metric, ideally at the base-question level.

8. DOCUMENT THE CHANGE
Update README / comments so it is explicit that:
- the experiment's main outcome is a continuous preference measure
- binary choice can hide or distort changes in preference strength
- the revised analysis aligns the statistical tests with the intended scientific claim

9. KEEP THE REFACTOR MINIMAL
Do not redesign the whole repo.
Preserve existing structure where possible.
Make the smallest clean change that fixes the mismatch between the main metric and the main analysis.

10. VALIDATION CHECKS
Add checks to ensure:
- the main summary really uses the continuous metric
- original/swapped are combined at the base-question level
- binary outputs are still available but clearly secondary
- main significance testing is no longer row-level binary-first by default

WHAT TO RETURN:
- exactly which files you changed
- where the original mismatch existed
- how the new continuous-metric-first analysis works
- example commands for generating:
  1. primary continuous-metric results
  2. secondary binary summaries

3. updated plots / tables

SUCCESS CRITERION:
After your changes, the experiment should have:
1. a primary continuous sycophancy metric
2. primary statistical tests aligned with that metric
3. base-question-level aggregation for paired original/swapped items
4. binary sycophancy results preserved only as secondary summaries

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:
Use base-level mean syc_logit as the default primary metric everywhere unless a function is explicitly marked as producing secondary binary summaries.

Constraint:
Prefer minimal invasive edits over a full rewrite. Reuse the existing file layout, naming style, and analysis flow whenever possible.

Metric and Stastical Issue misalignment desc

Issue 5 is: **your main metric and your main statistical tests are not fully aligned**.

You've been treating `syc_logit` as the main outcome, but parts of the analysis still lean too much on **binary choice results** like "picked the sycophantic answer or not." That creates a mismatch between what you say matters and what you mainly test.

Here's the core problem.

A **binary metric** throws away a lot of information. It only asks:

- did the model choose the sycophantic option?
- yes or no

But `syc_logit` asks something richer:

- how strongly did the model prefer the sycophantic option over the non-sycophantic one?

That difference matters a lot.

Example:

- Baseline: model prefers the sycophantic answer by a large margin.
- Steered: model still picks the sycophantic answer, but only by a tiny margin.

Under a binary metric, that looks like **no change**.
Under `syc_logit`, that is a **real improvement**, because the model became less strongly sycophantic.

The reverse can also happen:

- Baseline: barely picks sycophantic
- Steered: barely picks non-sycophantic

Binary says that is a full flip.
But the actual shift may be tiny and noisy.

So binary can both **miss real changes** and **overstate tiny ones**.

That is why issue 5 matters. If your scientific claim is really about shifting the model's preference structure, then the main statistical analysis should be built around the **continuous preference signal**, not mostly around thresholded yes/no outcomes.

Why this weakens the experiment:
A reviewer can say:

> your headline interpretation is about changing sycophantic preference, but your evidence is partly based on a coarser measure that discards the strength of that preference

and that is fair.

What you should do instead:

Make **base-level mean `syc_logit`** the primary statistic.
Then use:

- paired bootstrap CIs
- paired permutation test
- or paired Wilcoxon / t-test

at the **base-question level**, not the row level.

Keep **binary sycophancy rate** as a secondary result, because it is still interpretable and useful, but not the main inferential backbone.

So the clean setup is:

- **Primary metric:** base-level mean `syc_logit`
- **Primary tests:** tests on base-level `syc_logit`
- **Secondary metric:** binary sycophancy rate
- **Secondary tests:** binary comparisons if you want them

The short version:
Issue 5 is that you care about a **continuous preference measure**, but parts of the analysis still rely too much on a **coarser binary outcome**, which can hide or distort the real effect.

Row Pairs are not indepent

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The dataset contains paired rows for each base question:
- original
- swapped

These two rows are not independent observations. They are two versions of the same underlying question, created specifically to cancel answer-order bias.

The current analysis is too row-centric and risks treating these paired rows as if they were fully independent samples. That can make uncertainty estimates too optimistic and makes the analysis less faithful to the actual experiment design.

GOAL:
Refactor the analysis so the base question, not the raw row, is the primary analysis unit.

WHAT YOU NEED TO DO:

1. FIND THE PAIRING STRUCTURE
Locate where the code currently:
- stores or constructs original/swapped rows
- identifies the base question behind each pair
- computes row-level metrics and aggregates them

Make explicit how original and swapped are linked.

2. MAKE BASE QUESTION THE PRIMARY UNIT
Refactor the pipeline so the main analysis is performed at the base-question level.

For each base question:
- gather the original row result
- gather the swapped row result
- combine them into one base-level result
- use that base-level result in the main summaries and statistical tests

3. DO NOT TREAT ORIGINAL AND SWAPPED AS INDEPENDENT
Ensure the main analysis no longer behaves as if:
- original row = one independent sample
- swapped row = another independent sample

Instead, the pair should contribute one combined analysis unit.

4. PRESERVE ROW-LEVEL RESULTS AS AUXILIARY OUTPUTS
It is okay to keep row-level outputs for debugging, inspection, or sanity checks.

But:
- row-level outputs should be secondary
- the main summaries, coefficient selection, and statistical tests should be based on base-level aggregation

5. UPDATE METRIC AGGREGATION
Where the code currently computes averages, CIs, or tests over raw rows, update it so the default path is:
- aggregate to base question first
- then compute summary statistics across base questions

6. UPDATE ANY SPLITTING / SAMPLING LOGIC
If the code creates train/tune/test splits or subsamples, make sure it operates at the base-question level.

Original and swapped versions of the same base question must stay together in the same split.

7. UPDATE STATISTICAL TESTS
Modify the main statistical procedures so they operate over base-level values, not raw paired rows.

Examples:
- bootstrap over base questions
- paired comparisons at the base-question level
- base-level mean syc_logit as the main statistic

8. ADD VALIDATION CHECKS
Add checks to ensure:
- every base question has both original and swapped rows
- no pair is split across different partitions
- the main analysis is using the number of base questions, not the number of raw rows, as the effective unit count

9. UPDATE OUTPUTS AND DOCS
Make the outputs and README explicit that:
- original/swapped are paired
- pairing is used to cancel answer-order bias
- the correct independent unit is the base question
- row-level-only analysis can overstate certainty

10. KEEP THE REFACTOR MINIMAL
Do not redesign the whole repo.
Preserve the existing pipeline structure where possible.
Make the smallest clean change that makes the analysis faithful to the paired design.

WHAT TO RETURN:
- exactly which files you changed
- where the original row-level assumption existed
- how base-level aggregation now works
- example commands for running the updated analysis
- a short explanation of why this fixes the statistical weakness

SUCCESS CRITERION:
After your changes, the experiment should:
1. treat original/swapped as a linked pair
2. combine them into one base-level result
3. use base questions as the main analysis unit
4. avoid presenting the raw row count as if it were the true number of independent observations

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:
Base-level aggregation should be the default everywhere for main metrics, coefficient selection, confidence intervals, and final reported results unless a function is explicitly marked as row-level debugging output.

Row Pairs Desc

Issue 6 is: **you're treating row pairs like independent data points when they really belong to the same underlying question**.

Your setup has, for each base question, two rows:

- **original**
- **swapped**

Those are not two separate observations in the usual statistical sense. They are two versions of the **same item**, created specifically to cancel answer-order bias.

So the weakness is this:

What the experiment is doing conceptually

You made paired rows so that:

- if the model has an A-bias or B-bias,
- the original/swapped pair helps cancel that out.

That part is good.

Where the problem comes in

If, later, the analysis treats those two rows like fully independent samples, then the statistics are a little misleading.

Why? Because:

- the two rows share the same underlying content
- they are deliberately linked
- they are not two unrelated questions

So if you count them like separate independent datapoints, you are sort of pretending you have more independent evidence than you really do.

Simple example

Say you have **300 base questions**.

Because each has:

- original
- swapped

you now have **600 rows**.

But you do **not** really have 600 independent question units.
You have **300 paired units**.

If your stats behave like `n = 600` instead of something closer to `n = 300`, then:

- confidence intervals can get too tight
- p-values can look too strong
- effects can seem more certain than they really are

Why this matters specifically here

The whole purpose of the swapped version is to help neutralize positional bias.

That means the right thing to do is usually:

1. take the original result
2. take the swapped result
3. combine them into **one base-question-level score**
4. run the main analysis over those base-question scores

That respects the actual design.

Intuition

Think of it like testing the same student twice with the same question in slightly different formatting. You would not say that gives you two fully independent students' worth of evidence. It is still one underlying question unit.

What can go wrong if you ignore this

You can get results that look more statistically solid than they should.

Not because the effect is fake, but because the analysis is a little overconfident.

Clean fix

Make the **base question** the main analysis unit.

So for each base question:

- combine original + swapped
- compute one score
- then average and test across those combined scores

That way:

- the pairing does its job
- the inference matches the design
- the stats are cleaner

Short version

Issue 6 is:

you built paired data, but the main analysis may still be acting like those pairs are separate independent samples.

The fix is:

analyze at the base-question level, not the raw row level.

I can make the Claude Code prompt for issue 6 next.

Overfitting

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The experiment selects the “best” steering coefficient using a rule like:
- best improvement over random
- best margin relative to random control

However, this selection is still performed on the same evaluation dataset that is later used to report the final results.

This creates selection bias (winner’s curse), even though the selection rule is more sophisticated than simply picking the lowest raw score.

In other words:
- the experiment searches over many coefficients
- picks the one that looks best relative to random on this dataset
- then reports that same performance

This can produce overly optimistic results due to sample-specific noise.

GOAL:
Fix this issue by ensuring that coefficient selection (even when based on “improvement over random”) is performed only on a tuning split, and never on the held-out test set.

WHAT YOU NEED TO DO:

1. IDENTIFY CURRENT SELECTION LOGIC
Locate where the code:
- computes performance across multiple coefficients
- compares real vectors vs random control
- selects the “best” coefficient (e.g., max improvement over random)

Explain clearly where this selection is currently happening on the full dataset.

2. IMPLEMENT DATA SPLIT (IF NOT ALREADY DONE)
Ensure the evaluation data is split into:
- tuning split (for selection)
- test split (for final reporting)

CRITICAL:
- data contains paired original/swapped rows
- split must be done at the base-question level

- original and swapped must stay together

3. RESTRICT SELECTION TO TUNING DATA ONLY
Refactor the selection logic so:
- all coefficient sweeps happen on the tuning split
- all comparisons against random (mean or distribution) happen on tuning only
- the “best” coefficient is chosen using only tuning results

4. DEFINE A FIXED SELECTION RULE
Make the selection rule explicit and deterministic, for example:
- choose coefficient with best mean base-level syc_logit improvement over mean random control
- subject to degradation constraints

This rule must be:
- defined before test evaluation
- applied only to tuning data

5. FREEZE THE SELECTED COEFFICIENT
Once the best coefficient is selected on tuning:
- store it
- do not recompute or re-optimize on test data
- do not compare multiple coefficients on test

6. EVALUATE ON HELD-OUT TEST SET
Run evaluation using:
- the fixed selected coefficient
- the test split only

These results are the only ones used for final reporting.

7. PREVENT LEAKAGE
Ensure the code does NOT:
- recompute “best over random” on the test set
- search across coefficients on test
- use test results in any selection logic

8. UPDATE ANALYSIS OUTPUTS
Clearly separate:
- tuning results (used for selection)
- test results (used for reporting)

For clarity:
- show coefficient sweep plots for tuning

- show single-point results for test (no re-selection)

9. VALIDATION CHECKS
Add checks to ensure:
- selection is performed only on tuning data
- test evaluation uses exactly one fixed coefficient per condition
- no base question appears in both splits

10. DOCUMENT THE CHANGE
Update README/comments to explain:
- why “best over random” alone does not fix overfitting
- why selection must be separated from evaluation
- that final results come from held-out data only

WHAT TO RETURN:
- which files you modified
- where the original “best over random on full data” logic existed
- how the new selection pipeline works
- example commands for:
  1. running coefficient sweep on tuning split
  2. selecting best coefficient
  3. running final evaluation on test split

SUCCESS CRITERION:
After your changes, the workflow must be:

1. split data into tune/test
2. sweep coefficients on tuning split
3. select best coefficient (e.g., best over random) on tuning only
4. freeze the choice
5. evaluate once on test
6. report only test results as final

The key requirement:
“best over random” must NOT be computed on the same dataset used for final reporting.

Do not redesign the whole project. Make the minimal clean change that fixes this selection bias.

Overfitting desc

Issue 7 is: **even your “best over random” rule can still overfit if you use it on the same dataset you report on**.

This is basically a more specific version of issue 1.

You might think:

- “we’re not just picking the lowest coefficient”
- “we’re picking the coefficient that beats random the most”
- “so that’s safer”

It is safer than a naive rule, but it still has the same underlying weakness if it is done on the same eval set.

Why

Because the problem is not only **which rule** you use.

The problem is also **where** you use it.

If you look at one dataset and ask:

- which coefficient has the best sycophancy reduction?
  or
- which coefficient has the best margin over random?
  or
- which coefficient has the best tradeoff score?

you are still **searching over many options on the same sample** and then reporting the winner from that same sample.

So the winner can still look unusually good just because of sample-specific luck.

Simple example

Say on your eval set:

- `+1000` beats random by `0.08`

- `+2000` beats random by `0.21`
- `+3000` beats random by `0.10`

Then you pick `+2000` because it has the biggest margin over random.

That sounds reasonable. But maybe on a different sample the truth is more like:

- `+1000` beats random by `0.11`
- `+2000` beats random by `0.12`
- `+3000` beats random by `0.10`

So `+2000` was not really uniquely best. It just looked especially good on that one dataset.

Why this is “too convenient”

Because a smart-looking selection rule can give the impression that overfitting has been solved, when really it has just been **repackaged**.

Instead of:

- “pick the lowest score”

you are doing:

- “pick the best score relative to random”

But if both are chosen on the same eval set, both can still be optimistic.

What makes this different from issue 1

Issue 1 was the broad problem:

- selection and reporting happen on the same data

Issue 7 is the narrower point:

- even your more sophisticated **selection criterion** is still vulnerable if it is applied on the same data

So issue 7 is basically:
a better rule does not fix data leakage.

Why this matters

A reviewer could say:

> You say you avoided naive cherry-picking by selecting based on improvement over random, but you still chose the winner on the same benchmark you report, so the result may still be optimistic.

And they would be right.

Clean fix

Use a **predefined rule on tune only**.

For example:

- choose the coefficient with the best base-level mean `syc_logit` improvement over mean random control
- subject to degradation staying below a threshold

Then freeze that choice and evaluate once on held-out test.

That way:

- the rule is fixed
- the data used for choosing is separate
- the test result is honest

Short version

Issue 7 is:

“best over random” is a better selection rule than “best raw score,” but it still does not solve overfitting if you apply it on the same dataset you later use as proof.

Ancedotal Issues

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The current qualitative checks are too anecdotal. The repo includes illustrative examples of model behavior, but these examples are not systematic enough to support strong conclusions. A small number of handpicked outputs can help explain the result, but they do not establish whether the observed behavior is representative.

As a result, the current qualitative analysis is vulnerable to the criticism that it is cherry-picked or insufficiently rigorous.

GOAL:
Upgrade the qualitative analysis so it becomes a structured, reproducible evaluation rather than a loose collection of examples.

WHAT YOU NEED TO DO:

1. FIND THE CURRENT QUALITATIVE CHECKS
Locate where the code currently:
- samples or prints example outputs
- inspects qualitative behavior
- checks for overcorrection, weird outputs, or disagreement behavior

Explain clearly where the current qualitative analysis is anecdotal and why it is not strong enough.

2. CREATE A STRUCTURED QUALITATIVE EVAL
Refactor the pipeline so qualitative behavior is evaluated on a defined held-out set of prompts rather than only through ad hoc examples.

For each response, support labeling for categories such as:
- appropriate agreement
- appropriate disagreement
- hedging / neutrality
- refusal
- incoherence / unusable output
- overcorrection
- persona drift / weird role leakage

You may implement this with:
- rubric-based programmatic checks where feasible
- structured annotation files

- or LLM-judge / classifier support if the repo already has that pattern

Keep the design simple and reproducible.

3. MAKE THE QUALITATIVE SET HELD-OUT
The prompts used for qualitative analysis should be clearly separated from the prompts used for tuning decisions.

Do not rely only on examples pulled from the main evaluation set after looking at the results.

4. TURN QUALITATIVE BEHAVIOR INTO SUMMARY OUTPUTS
Add outputs that summarize how often each steering condition produces:
- appropriate disagreement
- inappropriate agreement
- hedging
- refusal
- incoherence
- overcorrection

Do not rely only on raw text examples.

5. KEEP EXAMPLES, BUT MAKE THEM SECONDARY
You should still preserve example outputs, but they should now serve as illustrations of measured patterns, not as the main evidence.

For example:
- include a few representative examples per category
- but back them with counts or rates from the structured rubric

6. MAKE CONDITIONS COMPARABLE
Ensure the same qualitative rubric is applied across:
- baseline
- real steering vectors
- random controls

This should make it possible to compare whether a vector reduces sycophancy by:
- healthy pushback
or
- hedging, refusal, or degradation

7. UPDATE ANALYSIS OUTPUTS
Produce outputs such as:
- per-condition category counts
- per-condition category rates

- optional confusion-style summary of response behavior
- a small curated example file linked to the rubric categories

8. DOCUMENT THE CHANGE
Update README / comments so it is explicit that:
- examples alone are not sufficient evidence
- the repo now includes a structured qualitative evaluation
- examples are used to illustrate the measured behavioral categories

9. KEEP THE REFACTOR MINIMAL
Do not redesign the whole repo.
Preserve the current qualitative scripts where possible.
Convert them into a cleaner structured evaluation with minimal invasive edits.

10. VALIDATION CHECKS
Add checks to ensure:
- the qualitative evaluation runs on a defined prompt set
- outputs include category summaries, not just raw examples
- examples shown in reports are traceable to the structured evaluation categories

WHAT TO RETURN:
- exactly which files you changed
- where the original anecdotal qualitative logic existed
- how the new structured qualitative evaluation works
- example commands for:
  1. running the qualitative eval
  2. generating category summaries
  3. exporting representative examples

SUCCESS CRITERION:
After your changes, the repo should no longer rely on a few handpicked examples as its qualitative evidence.

Instead, it should:
1. evaluate qualitative behavior on a defined held-out set
2. label responses with a structured rubric
3. summarize behavior by category and condition
4. use examples only as illustrations of systematic results

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:

Representative examples shown in outputs should be sampled from rubric-labeled categories, not manually cherry-picked after inspection.

Ancedtoal Issues desc

Issue 8 is: **your qualitative checks are too anecdotal, so they help illustrate the result but do not really prove it**.

Right now, qualitative examples are useful for showing what the model is doing, but a few examples can always be dismissed as cherry-picked. The problem is not that qualitative analysis is bad. The problem is that it is currently too informal.

What the weakness is

You likely have things like:

- a few nice examples where `skeptic` pushes back well
- a few examples of overcorrection
- a few weird outputs

That helps intuition, but it does not tell you:

- how often good pushback happens
- how often bad pushback happens
- whether one steering condition is consistently better than another

So the evidence is descriptive, not systematic.

Why this matters

A reviewer can always say:

> these examples are interesting, but how do I know they are representative?

And that is fair. A handful of examples can make almost any method look good.

Example

Suppose you show 3 outputs:

- one where baseline agrees too much
- one where skeptic pushes back correctly

- one where random gets weird

That is nice, but it does not answer:

- does skeptic usually improve things?
- does it often overcorrect?
- does it increase refusals?
- is `judge` better than `skeptic`, or did you just show the nicest skeptic example?

The deeper issue

Your experiment is not only about whether the metric changes. It is also about **how** behavior changes.

A lower sycophancy score could come from:

- better disagreement
- more hedging
- more refusals
- more incoherence
- more generic caution

Qualitative analysis is exactly where you should separate those, but only if it is done in a structured way.

What the fix is

Turn the qualitative checks into a **small rubric-based evaluation**.

For a held-out set of prompts, label each response for things like:

- appropriate agreement
- appropriate disagreement

- hedge / neutrality
- refusal
- incoherence
- overcorrection
- persona drift

Then you can still include examples, but the examples are now backed by counts or rates.

What that gives you

Instead of saying:

> here are some examples showing skeptic looks better

you can say:

> on a held-out qualitative set, skeptic produced more appropriate disagreement and less blind agreement than baseline, with X% overcorrection and Y% refusal

That is much stronger.

Short version

Issue 8 is:

your examples are illustrative, but not systematic enough to count as strong evidence.

The fix is:

use a structured rubric on a held-out set, then use examples only to illustrate the measured pattern.

Issues with claiming steering working internally

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The current internal / mechanistic analysis is too thin. The code appears to use a narrow projection-style probe (for example, projection onto a steering direction or axis at a single layer and/or a single token position) and then uses that as evidence for how steering works internally.

That is not strong enough to support a real mechanism claim.

WHY THIS IS A PROBLEM:
A single projection snapshot can be:
- too local
- too noisy
- not representative of the full generation process
- weakly connected to the observed behavioral effect

In particular, if the current analysis only looks at something like the last prompt token at one layer, it may miss the more important internal changes that happen during response generation.

So the current analysis may show:
- behavior changed
- one projection changed

but it does not strongly establish:
- where the internal change happens
- whether it persists during generation
- whether it tracks the behavioral effect
- whether the claimed representation shift is actually the mechanism

GOAL:
Strengthen the mechanistic analysis so it better links internal representation changes to observed behavioral changes.

WHAT YOU NEED TO DO:

1. FIND THE CURRENT PROJECTION / MECHANISM LOGIC
Locate where the code currently:
- computes projection onto the Assistant Axis or steering vector
- chooses token position(s)
- chooses layer(s)
- summarizes internal changes

Explain clearly what the current mechanism analysis is doing and why it is too narrow.

2. EXPAND TOKEN-LEVEL ANALYSIS
Refactor the code so internal projections can be measured not only at one prompt token, but also across generated response tokens.

At minimum, support:
- prompt-final-token projection
- response-token projections
- average projection across the generated response
- optional per-token trajectory outputs

The goal is to see whether the internal shift persists during generation, not just at one input position.

3. EXPAND LAYER-LEVEL ANALYSIS
Do not rely only on one single layer unless the codebase already has a very strong reason for doing so.

Add support for:
- analyzing a small layer range around the target layer
- comparing projection behavior across nearby layers
- summarizing where the internal shift is strongest

This does not need to become a giant layer sweep. Keep it minimal but more informative than one isolated probe.

4. LINK INTERNAL SHIFT TO BEHAVIOR
Add analysis that directly compares:
- magnitude of internal projection shift
with
- magnitude of behavioral change (for example syc_logit reduction)

Useful outputs:
- per-condition correlation between projection shift and behavioral shift
- plots showing whether stronger internal movement corresponds to stronger reduction in sycophancy
- summaries comparing baseline vs steered conditions on both internal and external metrics

The point is to make the mechanism story behavior-linked, not just descriptive.

5. SEPARATE SUGGESTIVE VS STRONGER EVIDENCE
In the updated outputs and docs, clearly distinguish:
- descriptive projection summaries

from
- stronger evidence connecting internal changes to behavior

Do not overclaim from a single projection value.

6. UPDATE OUTPUTS
Produce outputs that make the mechanism analysis more interpretable, such as:
- average response-token projection by condition
- projection trajectories over generated tokens
- projection by layer
- relationship between projection shift and behavioral effect

Keep existing outputs where useful, but expand them so the mechanism story is not based on one narrow number.

7. KEEP THE REFACTOR MINIMAL
Do not redesign the entire repo.
Preserve current behavior where possible.
Add the smallest clean extension that upgrades the internal analysis from a thin probe to a more convincing mechanism analysis.

8. DOCUMENT THE CHANGE
Update README / comments so it is explicit that:
- a single-token, single-layer projection is only limited evidence
- the revised analysis examines projections during generation
- the new analysis better supports claims about how steering works internally

9. VALIDATION CHECKS
Add checks to ensure:
- response-token projection analysis actually runs on generated outputs
- multi-layer outputs are consistent with selected layer indices
- mechanism outputs are saved separately from main behavioral metrics
- plots/tables clearly label whether they are prompt-token or response-token analyses

WHAT TO RETURN:
- exactly which files you changed
- where the original thin projection analysis existed
- how the new mechanism analysis works
- example commands for:
  1. running prompt-token projection analysis
  2. running response-token projection analysis
  3. generating layer-wise / token-wise plots
  4. comparing projection shifts to behavioral shifts

SUCCESS CRITERION:
After your changes, the codebase should no longer rely on a single narrow projection snapshot as the main mechanism evidence.

Instead, it should make it possible to show:
1. where internal representation shifts occur
2. whether they persist during response generation
3. how they vary across nearby layers
4. whether they track the observed behavioral effect

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:
Do not present prompt-final-token projection alone as sufficient mechanism evidence; treat it as one view among several, with response-token analysis included by default in the expanded mechanism workflow.

Issues with claiming steering descprtion

Issue 9 is: **your projection/mechanism analysis is too thin, so the paper can't strongly claim *how* the steering is working internally**.

Right now, you likely have some internal readout like "projection onto the Assistant Axis at one layer / one token position." That is useful, but it is still a pretty weak mechanism story.

Why that's a problem:

Your paper is not just saying:

- behavior changed

It is also implicitly saying something like:

- behavior changed **because** the intervention moved the model along a meaningful internal direction

That second claim needs stronger evidence than one narrow probe.

The core weakness is that a single projection snapshot can be:

- too local
- too noisy
- not representative of the whole response process

For example, if you only look at the **last prompt token** at one layer, you may miss what actually matters during **generation**. A lot of the relevant internal change may show up while the model is producing the answer, not just right before it starts.

So the danger is this:

- you observe a behavioral effect
- you observe some projection shift
- but the link between them is still a bit hand-wavy

A reviewer could say:

> nice behavioral result, but the internal analysis is too shallow to support a strong mechanism claim

and that would be fair.

A simple example:
suppose steering reduces sycophancy, and you show that the final prompt token projects less onto the Assistant Axis. That is interesting. But it still doesn't tell us clearly:

- whether the whole response trajectory moved
- whether the effect is stable across tokens
- whether the key change happens only at one layer
- whether this internal shift actually tracks the behavioral improvement

So the analysis is suggestive, but not very deep.

The clean fix is to make the internal analysis more behavior-linked. For example:

- measure projection over **response tokens**, not just one prompt token
- compare baseline vs steered across the whole generated answer
- maybe inspect a small set of nearby layers, not just one
- test whether bigger internal shifts actually correlate with bigger behavioral shifts

That gives you a much stronger story:

- the vector changes internal representation
- that change persists during generation
- and it tracks the reduction in sycophancy

So the short version is:

Issue 9 is that your internal/mechanistic evidence is currently too narrow to strongly support the claim that the steering works through a meaningful representation shift, rather than just producing an observed behavioral effect with a thin post hoc probe.

Too dependent on one sample

You are reviewing and modifying an ML experiment codebase.

There is a design weakness in the current experiment:

ISSUE:
The current results rely too heavily on a single sample / single run. The main conclusions appear to come from one particular subset, one split, one seed, or one pass through the pipeline.

This means the reported effect may be real, but it is not yet clear how stable it is.

WHY THIS IS A PROBLEM:
A result can look stronger or cleaner than it really is if it depends too much on:
- one random split
- one sampled subset of questions
- one seed
- one random-control draw
- one particular evaluation run

Without repeated resampling or repeated runs, it is hard to tell whether the finding is:
1. robust and stable
or
2. unusually favorable on this one sample

GOAL:
Strengthen the experiment so the main conclusions are supported by repeated splits / repeated runs, not just one sample.

WHAT YOU NEED TO DO:

1. FIND WHERE THE CURRENT PIPELINE IS SINGLE-RUN
Locate where the code currently:
- creates the eval subset or split
- fixes the seed
- runs one tuning/evaluation pass
- reports final results from one run

Explain clearly where the experiment is currently too dependent on one sample.

2. ADD REPEATED RESAMPLING / REPEATED RUN SUPPORT
Refactor the pipeline so it can run multiple repetitions of the experiment.

Support at least one of these cleanly:
- multiple random tune/test splits

- multiple seeds
- repeated subsamples
- cross-validation at the base-question level

Preferred default:
- 3 to 5 repeated tune/test splits at the base-question level

3. KEEP THE BASE-QUESTION STRUCTURE INTACT
The dataset contains paired rows:
- original
- swapped

These must stay together in any repeated split or resample.
Do not split paired rows apart.

4. RUN THE FULL SELECTION + EVALUATION LOOP PER REPEAT
For each repeat:
- create or load the split/subsample
- run tuning / coefficient selection on the tune portion
- freeze the selected setting
- evaluate on the held-out test portion

Do not just reuse one chosen coefficient across all repeats unless explicitly requested.

5. AGGREGATE RESULTS ACROSS REPEATS
Update the analysis so final reported results include stability information across repeated runs.

Examples:
- mean effect across repeats
- standard deviation across repeats
- confidence interval across repeats
- distribution of selected coefficients
- consistency of ranking across conditions

6. MAKE INSTABILITY VISIBLE
Add outputs that show whether the result is stable or fragile.

Useful summaries:
- how often each condition wins across repeats
- how much selected coefficients vary
- how much effect sizes vary
- whether the main conclusion survives across repeats

7. KEEP SINGLE-RUN OUTPUTS AVAILABLE

Preserve the ability to run one fast experiment for debugging.

But the main reported workflow should support repeated evaluation, and the README should make clear that single-run results are not sufficient for the strongest claims.

8. UPDATE DOCS
Update README / comments so it is explicit that:
- one split / one seed is not enough to demonstrate robustness
- repeated runs provide evidence that the result is stable
- final conclusions should be based on aggregated repeated-run results, not just one favorable run

9. KEEP THE REFACTOR MINIMAL
Do not redesign the whole repo.
Preserve the current structure where possible.
Make the smallest clean change that adds repeated-run robustness evaluation.

10. VALIDATION CHECKS
Add checks to ensure:
- repeated runs use distinct seeds or distinct splits when expected
- original/swapped pairs stay together
- aggregation is over repeated runs, not just raw rows
- outputs clearly distinguish per-run results from aggregated robustness summaries

WHAT TO RETURN:
- exactly which files you changed
- where the original single-sample weakness existed
- how the repeated-run workflow now works
- example commands for:
  1. running one repeat
  2. running multiple repeats
  3. aggregating repeat-level results
  4. generating robustness summaries / plots

SUCCESS CRITERION:
After your changes, the experiment should no longer rely on one sample / one run as the only evidence.

Instead, it should make it possible to show:
1. whether the effect survives across repeated splits or seeds
2. how much the effect size varies
3. whether the chosen coefficients / winners are stable
4. whether the main conclusion is robust rather than lucky

Do not stop at describing the issue. Implement the fix cleanly in the codebase and show the updated workflow.

Additional requirement:
Use repeated base-question-level tune/test splits by default, and report aggregated held-out test results across repeats as the main robustness summary.

Dependent Desc

Issue 10 is: **the experiment is too dependent on one sample / one run, so you do not yet know how stable the result really is**.

Right now, if most of your conclusion comes from:

- one selected subset of questions
- one split
- one seed
- one pass through the pipeline

then the result might be real, but it could also be **fragile**.

What the weakness is

Even a good method can look stronger or weaker depending on:

- which questions ended up in the sample
- which tune/test split you happened to use
- which random seed was used
- which random control draw you got

So the issue is not necessarily bias in the code. It is **lack of robustness evidence**.

Why this matters

A reviewer can ask:

> If I rerun this with a different split or different subset, do I get basically the same conclusion?

If the answer is “not sure,” then the paper looks less solid.

Simple example

Suppose on your current sample:

- skeptic beats axis clearly

- judge is second best
- +2000 looks best

But on another equally reasonable split:

- skeptic still helps, but by less
- judge and skeptic are closer
- +1000 and +2000 are basically tied

That would not kill the result, but it would show the effect is less stable than the single reported run suggests.

What can go wrong if you ignore this

You may accidentally overstate:

- effect size
- certainty
- ranking between conditions
- confidence in the chosen coefficient

Again, the result may still be true. The problem is that one sample can make it look cleaner than it really is.

Clean fix

Show that the result holds across **multiple resamplings**.

Good options:

- multiple random tune/test splits
- multiple seeds
- repeated subsamples

- cross-validation at the base-question level

You do not need all of them. Even **3–5 repeated splits** would help a lot.

Best way to think about it

Issue 10 asks:

Is this a stable effect, or just a good-looking run?

That is different from issue 1:

- issue 1 = leakage / selection bias
- issue 10 = stability / robustness across reruns

Short version

Issue 10 is:

your result may be too tied to one particular sample and one particular run, so you need repeated splits/seeds to show the finding is robust rather than lucky.

If you want, I'll make the Claude Code prompt for issue 10 next.

Master Prompt

You are modifying an ML experiment codebase.

Your task is to fix core experimental design flaws and upgrade the experiment into a clean, workshop-quality study.

IMPORTANT:
- Make minimal, clean, consistent changes.
- Do NOT redesign the entire project.
- Do NOT introduce conflicting logic.
- All changes must follow a single coherent pipeline.

CORE REQUIREMENTS (NON-NEGOTIABLE)

You MUST enforce these invariants everywhere:

1. Base question is the unit of analysis (NOT rows)
2. Original + swapped pairs are always kept together
3. Tuning and testing are strictly separated
4. Selection NEVER uses test data
5. Continuous metric (e.g., syc_logit) is primary
6. Binary metrics are secondary only
7. Real vectors must be compared to MULTIPLE random vectors
8. Improvements must be checked against degradation
9. Results must generalize beyond one eval format

IMPLEMENTATION (IN ORDER — DO NOT REORDER)

==========
STEP 1 — BASE-LEVEL DATA MODEL
==========

Refactor the data handling so:

- Each base question is the primary unit
- Each base question has exactly:
  - one original row
  - one swapped row

Create a base-level aggregation function:

For each base question:
- combine original + swapped into ONE score
- store base_id → aggregated result

ALL downstream analysis MUST use base-level values.

Row-level values may exist ONLY for debugging.

Add validation:
- every base_id has exactly 2 entries
- no missing pairs

==========
STEP 2 — TUNE / TEST SPLIT
==========

Split dataset into:
- tuning set
- held-out test set

CRITICAL:
- split at base_id level
- NEVER split original/swapped pairs

Default:
- 50% tune / 50% test
- fixed random seed

Outputs:
- eval_tune.json
- eval_test.json

Add validation:
- no base_id overlap between splits

==========
STEP 3 — MULTI-RANDOM CONTROL
==========

Replace single random vector with N random vectors.

Requirements:
- N = 5–10
- same dimensionality as real vectors

- same normalization
- same layer
- same coefficient sweep

Store results per random vector.

Compute:
- mean random performance
- variance / spread

ALL comparisons must use:
- mean(random_vectors)
NOT a single random vector

==========
STEP 4 — COEFFICIENT SWEEP (TUNING ONLY)
==========

Run full sweep ONLY on tuning set.

For each condition + coefficient:
- compute base-level mean syc_logit
- compute degradation metrics
- compare vs mean random control

Selection rule (must be deterministic):
- minimize base-level mean syc_logit
- subject to degradation thresholds
- optionally maximize margin over mean random

Select ONE coefficient per condition.

SAVE selected coefficients.

DO NOT:
- use test set here
- reselect later

==========
STEP 5 — FREEZE SELECTION
==========

After tuning:
- lock selected coefficient

- no further optimization allowed

Add assertion:
- test phase cannot run multiple coefficients

==========
STEP 6 — TEST EVALUATION
==========

Run evaluation on test set using:
- ONLY the selected coefficient

Compute:
- base-level mean syc_logit (PRIMARY)
- binary syc rate (SECONDARY)

DO NOT:
- sweep coefficients
- compare multiple coefficients
- compute “best” anything

Test is evaluation only.

==========
STEP 7 — DEGRADATION CHECKS
==========

Add degradation metrics:

Required:
- refusal rate
- hedge / noncommittal rate
- response length
- choice confidence / margin
- invalid / incoherent output rate

Optional (if easy):
- small QA or reasoning set

Outputs must separate:
- benefit (syc reduction)
- cost (degradation)

Selection must:

- reject clearly degraded settings
OR
- report tradeoffs explicitly

==========
STEP 8 — SECOND EVAL FAMILY (FREE RESPONSE)
==========

Add new eval:

Free-response disagreement:
- user gives opinion/claim
- model responds in natural language

Label responses:
- agree
- disagree
- hedge
- refuse
- incoherent

Run this for:
- baseline
- real vectors
- random controls

Goal:
Verify effect is NOT limited to A/B format.

==========
STEP 9 — METRIC ALIGNMENT
==========

Primary metric:
- base-level mean syc_logit

Primary stats:
- base-level aggregation
- paired methods if needed

Binary metrics:
- secondary only
- must NOT drive conclusions

Add assertion:
- all main summaries use continuous metric

==========
STEP 10 — OUTPUT STRUCTURE
==========

Produce clearly separated outputs:

TUNING:
- coefficient sweep
- selected coefficients

TEST:
- final results (continuous primary)
- binary secondary

RANDOM CONTROL:
- distribution summary

DEGRADATION:
- refusal / hedge / coherence metrics

FREE RESPONSE:
- behavior breakdown

==========
STEP 11 — VALIDATION CHECKS
==========

Add strict checks:

- no base_id in both tune and test
- each base_id has exactly 2 rows
- original/swapped never split
- selection only happens on tune
- test uses exactly 1 coefficient
- multiple random vectors are used
- base-level aggregation is used everywhere

Fail loudly if violated.

==========
STEP 12 — DOCUMENTATION

==========

Update README:

Explain clearly:
- base-level analysis
- tune/test separation
- multi-random control
- degradation checks
- multi-eval design

SUCCESS CRITERIA

Final experiment MUST:

1. Separate tuning from evaluation
2. Use base-question as unit
3. Compare against multiple random vectors
4. Distinguish real improvement vs degradation
5. Show generalization beyond forced-choice

WHAT TO RETURN

- Files modified
- Explanation of each fix
- New workflow
- Commands for:
  1. dataset build
  2. tuning run
  3. coefficient selection
  4. test evaluation
  5. free-response eval
  6. final analysis

Do NOT stop at describing issues. Implement everything cleanly and consistently.